University of Engineering and Technology Lahore

Department of Computer Science
Course: Parallel and Distributed Computing
Instructor: Waqas Ali

Lab 2: Point-to-Point Communication in MPI

1. Objective

- Understand how MPI processes execute the same binary but differentiate their work using rank.
- Learn the syntax and semantics of blocking point-to-point communication: `MPI_Send` and `MPI_Recv`.
- Explore the use of tags and wildcards (`MPI_ANY_TAG`, `MPI_ANY_SOURCE`).
- Implement a parallel sum program using master-slave communication.
- Measure execution time and compute speedup for different numbers of processes.
- Implement a custom gather operation using `MPI_Send` and `MPI_Recv`.

---

2. Theory

2.1 MPI Execution Model Revisited

When an MPI program runs, **all processes execute the same compiled binary**. This is the **SPMD** (Single Program, Multiple Data) model. So what makes a parallel program different from running the same program multiple times?

Two key features distinguish a parallel MPI program:

1. **Conditional execution based on rank**
   Each process obtains its unique rank (via `MPI_Comm_rank`) and uses it to decide which part of the code to execute. Code that is not guarded by a rank-specific condition will be executed by **all** processes.
2. **Explicit communication**

Processes exchange data by sending and receiving messages. This is what allows them to cooperate on a single task.

2.2 Point-to-Point Communication: `MPI_Send` and `MPI_Recv`

The fundamental communication operations in MPI are sending and receiving messages between two processes.

`MPI_Send` – Blocking Send

```
int MPI_Send(const void *buf, int count, MPI_Datatype datatype, int
dest,int tag, MPI_Comm comm);
```

- `buf`: starting address of the send buffer.
- `count`: number of elements to send.
- `datatype`: type of each element (e.g., `MPI_INT`, `MPI_DOUBLE`).
- `dest`: rank of the destination process.
- `tag`: message tag (an integer between 0 and 32767) used to identify the message.
- `comm`: communicator (usually `MPI_COMM_WORLD`).

Behaviour: `MPI_Send` is **blocking**. It does not return until the send buffer can be safely reused – either because the message has been copied to a system buffer or because it has been handed off to the network. This means the sender cannot proceed until the send operation is complete (or at least until the buffer is free).

`MPI_Recv` – Blocking Receive

```
int MPI_Recv(void *buf, int count, MPI_Datatype datatype, int
source,int tag, MPI_Comm comm, MPI_Status *status);
```

- `buf`: starting address of the receive buffer.
- `count`: maximum number of elements to receive.

- `datatype`: type of each element.
- `source`: rank of the sending process (can be `MPI_ANY_SOURCE`).
- `tag`: message tag (can be `MPI_ANY_TAG`).
- `comm`: communicator.
- `status`: structure that holds information about the received message (sender rank, tag, etc.). Use `MPI_STATUS_IGNORE` if not needed.

Behaviour: `MPI_Recv` is **blocking**. It does not return until a matching message has been received and copied into the receive buffer.

2.3 Wildcards

- `MPI_ANY_SOURCE`: Allows receiving from any sender. The actual sender rank can be retrieved from `status.MPI_SOURCE`.
- `MPI_ANY_TAG`: Allows receiving with any tag. The actual tag can be retrieved from `status.MPI_TAG`.

Important: Wildcards can only be used in `MPI_Recv`. In `MPI_Send`, the destination rank must be explicitly specified.

2.4 Matching Rules

A send and a receive match if:

- They use the same communicator.
- The receiver specifies the sender's rank, or uses `MPI_ANY_SOURCE`.
- The receiver specifies the send's tag, or uses `MPI_ANY_TAG`.
- The datatypes are compatible (they don't have to be identical, but must be convertible; in practice we keep them the same).

2.5 Example: Simple Send/Receive

The following program demonstrates a send from process 2 to process 4. (We assume at least 5 processes are available.)

```c
#include <stdio.h>
#include <stdlib.h>
#include <mpi.h>

int main(int argc, char *argv[]) {
    int mynode, totalnodes;
    int sender = 2;
    int receiver = 4;
    int tag = 123;
    MPI_Status status;

    MPI_Init(&argc, &argv);
    MPI_Comm_size(MPI_COMM_WORLD, &totalnodes);
    MPI_Comm_rank(MPI_COMM_WORLD, &mynode);

    // For simplicity, we fix datasize
    int datasize = 10;
    double *databuffer = (double*) malloc(datasize *
sizeof(double));

    if (mynode == sender) {
        // Initialize some data
        for (int i = 0; i < datasize; i++)
            databuffer[i] = i * 1.0;
        MPI_Send(databuffer, datasize, MPI_DOUBLE, receiver, tag,
MPI_COMM_WORLD);
        printf("Process %d sent data to %d\n", mynode, receiver);
    }

    if (mynode == receiver) {
        MPI_Recv(databuffer, datasize, MPI_DOUBLE, sender, tag,
MPI_COMM_WORLD, &status);
        printf("Process %d received data from %d\n", mynode,
status.MPI_SOURCE);
        // Print received data
        for (int i = 0; i < datasize; i++)
            printf("%f ", databuffer[i]);
        printf("\n");
    }
```

```
    free(databuffer);
    MPI_Finalize();
    return 0;
}
```

Compile and run (with at least 5 processes):

```
mpicc -o send_recv send_recv.c
mpirun -np 5 ./send_recv
```

Observe that only processes 2 and 4 print messages; others do nothing.

---

3. Parallel Sum of Numbers from 1 to 1000

This classic example distributes the work of summing numbers from 1 to 1000 across all processes. Each process computes a partial sum, then all non-zero processes send their partial sum to process 0, which accumulates the total.

3.1 Algorithm

- Process 0 is the **master**; all others are **workers**.
- Each process determines its range of numbers based on its rank and total number of processes.
- Each process computes its local sum.
- Workers send their local sum to process 0.
- Process 0 receives all partial sums, adds them to its own, and prints the final total.

3.2 Code – `parallel_sum.c`

```
#include <stdio.h>
#include <mpi.h>

int main(int argc, char *argv[]) {
    int mynode, totalnodes;
```

```
    int sum, startval, endval, accum;
    MPI_Status status;
    double start_time, end_time;

    MPI_Init(&argc, &argv);
    MPI_Comm_size(MPI_COMM_WORLD, &totalnodes);
    MPI_Comm_rank(MPI_COMM_WORLD, &mynode);

    // Start timing after initialization (optional: synchronize
with barrier)
    MPI_Barrier(MPI_COMM_WORLD);
    start_time = MPI_Wtime();

    sum = 0;
    // Determine the range of numbers for this process
    startval = 1000 * mynode / totalnodes + 1;
    endval   = 1000 * (mynode + 1) / totalnodes;

    for (int i = startval; i <= endval; i++) {
        sum += i;
    }

    if (mynode != 0) {
        // Worker sends its partial sum to process 0
        MPI_Send(&sum, 1, MPI_INT, 0, 1, MPI_COMM_WORLD);
    } else {
        // Master receives from all workers
        for (int j = 1; j < totalnodes; j++) {
            MPI_Recv(&accum, 1, MPI_INT, j, 1, MPI_COMM_WORLD,
&status);
            sum += accum;
        }
        // Master now has the total sum
    }

    end_time = MPI_Wtime();

    if (mynode == 0) {
        printf("The sum from 1 to 1000 is: %d\n", sum);
        printf("Execution time with %d processes: %f seconds\n",
```

```
                totalnodes, end_time - start_time);
    }

    MPI_Finalize();
    return 0;
}
```

3.3 Discussion

- **Range calculation:** The formula `1000 * mynode / totalnodes + 1` ensures a (mostly) even distribution of numbers. Because integer division truncates, the first few processes may get one extra number if 1000 is not divisible by `totalnodes`. This is acceptable for this exercise.
- **Tag usage:** Both send and receive use tag `1`. This distinguishes this communication from any other messages that might be added later.
- **Blocking behaviour:** The master's receive loop will block until each worker sends its partial sum. The workers send and then finish.
- **Timing:** We placed `MPI_Barrier` before starting the timer to ensure all processes are ready. The barrier adds a small overhead but makes the timing more consistent.

---

4. Exercises | Everyone Spent 60 mins

Exercise 1: Parallel Sum with Timing and Speedup

1. **Implement** the above `parallel_sum.c` program.
2. **Run** it on 1, 2, 4, 8, and 16 processes (if available). If your system has fewer cores, use as many as possible and note the limitations.
3. **Record** the execution time for each run in a table.
4. **Compute** speedup = T(1) / T(N) for each N.
5. **Plot** speedup vs. number of processes. Include a line for ideal speedup.
6. **Explain** the observed speedup. Is it linear? Why or why not? (Consider communication overhead, load imbalance, and Amdahl's law.)

Exercise 2: Custom Gather – Sending Arrays to Process 0

Write an MPI program where:

- Process 0 is the master.
- Every other process (rank 1 to `size-1`) generates an array of **10 integers** (e.g., fill with `rank*100 + i` for i=0..9).
- Each worker sends its entire array to process 0.
- Process 0 receives each array and prints them, clearly indicating which process sent which data.

Requirements:

- Use point-to-point communication (`MPI_Send`/`MPI_Recv`) – **do not** use `MPI_Gather` (that will be covered later).
- Process 0 must allocate sufficient receive buffer(s). You may receive into a temporary buffer for each worker and print immediately, or store all data in a large array.
- Use tags to distinguish messages from different workers (optional, but good practice).
- Print the received data in a readable format.

Example output:

```
Process 0 received from 1: 100 101 102 ... 109
Process 0 received from 2: 200 201 202 ... 209
...
```

Hints:

- Process 0 does not know the order in which messages arrive. It can loop over all expected ranks and receive in any order, but using `MPI_ANY_SOURCE` and checking `status.MPI_SOURCE` is also possible.
- Remember to allocate enough space for the receive buffer.

Exercise 3: Investigate Wildcards

Modify the program from Exercise 2 so that process 0 uses `MPI_ANY_SOURCE` to receive messages. After each receive, print the source rank and the data. How does the output order compare with receiving in a fixed order? Run multiple times – is the order deterministic? Discuss.

Exercise 4: (Optional) Compute π Using Point-to-Point

In Lab 1, you used `MPI_Reduce` to compute π. Now implement the same π computation using only `MPI_Send` and `MPI_Recv`. Process 0 distributes the number of intervals, each process computes its partial sum, and then workers send their partial sums to process 0 which accumulates. Compare the code complexity and performance with the `MPI_Reduce` version.

---

5. Common Pitfalls

- **Deadlocks:** If two processes both send before receiving (cyclic dependency), the program may hang. Always design communication so that at least one process receives first, or use non-blocking communication (later).
- **Buffer overflow:** The receive buffer must be at least as large as the incoming message. If you specify a smaller `count` than the sender used, the receive will be truncated and an error may occur (or the program may crash).
- **Mismatched tags:** If tags don't match, the message will not be received and the program may hang.
- **Not all processes participate:** If one process does not call `MPI_Recv` when expected, the sender will block forever.

---

6. Deliverables

Prepare a PDF report containing:

1. **Screenshots** of successful runs for Exercise 1 (at least with 4 processes) and Exercise 2.
2. **A table** of execution times and speedup for Exercise 1.

3. **A speedup plot** for Exercise 1.
4. **Source code** for Exercise 2 (copy into the report).
5. **Answers** to any discussion questions (e.g., explanation of speedup, wildcard behaviour).
6. **A brief reflection** (10–15 sentences) on the challenges of point-to-point communication compared to using collective operations (like `MPI_Reduce`).

---

7. Further Reading

- MPI Tutorial – Point-to-point Communication
- LLNL MPI Tutorial – Point-to-point
- Book | parallel-programming-in-c-with-mpi-and-openmp. | chapter 4

In this lab you have mastered the fundamentals of MPI point-to-point communication. These skills are essential for building more complex parallel algorithms. Next, we will explore collective operations like scatter, gather, and allreduce.

Lab 2 Assessment Rubric

This rubric outlines how your lab report will be evaluated. Read it carefully to ensure you meet all requirements.

Total Points: 100

1. Exercise 1: Parallel Sum with Timing and Speedup (30 points)

CRITERION	EXCELLENT (30–27)	GOOD (26–22)	SATISFACTORY (21–18)	NEEDS IMPROVEMENT (17–0)
Code correctness	Program compiles without warnings, runs correctly on all tested process counts, and produces the correct sum.	Program compiles and runs correctly on most process counts; minor issues with edge cases.	Program compiles but fails on some process counts or produces incorrect sum.	Program does not compile or consistently produces wrong results.
Timing & speedup table	Clear table with execution times and speedup for all required process counts (1,2,4,8,16 if available).	Table present but missing one data point or slightly unclear.	Table incomplete or not properly labelled.	No table or completely incorrect.
Speedup plot	Neat, properly labelled plot with ideal speedup line; axes clearly marked.	Plot present but lacks ideal line or labels are incomplete.	Plot present but difficult to interpret.	No plot.
Analysis	Thoughtful explanation of speedup behaviour, referencing Amdahl's law, communication overhead, and load imbalance.	Explanation provided but lacks depth or misses one key factor.	Explanation superficial or contains errors.	No explanation.

2. Exercise 2: Custom Gather – Sending Arrays to Process 0 (30 points)

CRITERION	EXCELLENT (30–27)	GOOD (26–22)	SATISFACTORY (21–18)	NEEDS IMPROVEMENT (17–0)
Code correctness	Program compiles without warnings; all workers correctly generate and send arrays; process 0 receives and prints all data without errors.	Program works but may have minor issues (e.g., prints in wrong order, buffer size slightly off).	Program compiles but fails to receive all data or crashes for some process counts.	Program does not compile or major logical errors.
Use of point-to-point	Correct use of `MPI_Send`/`MPI_Recv` with appropriate tags and buffer management.	Mostly correct, but minor misuse (e.g., ignoring status, tag confusion).	Significant errors in send/receive usage.	No send/receive or completely incorrect.
Output clarity	Printed output clearly shows which process sent which data; formatting is neat.	Output present but slightly messy or hard to follow.	Output incomplete or confusing.	No output or completely garbled.

3. Exercise 3: Investigate Wildcards (Optional, 10 bonus points)

CRITERION	EXCELLENT (10)	GOOD (8–9)	SATISFACTORY (6–7)	NEEDS IMPROVEMENT (0–5)
Implementation	Successfully modifies Exercise 2 to use `MPI_ANY_SOURCE`; code works correctly.	Modification works but has minor issues.	Attempt made but not fully functional.	No attempt or completely broken.
Observation & explanation	Clearly explains whether the receive order is deterministic and why (or why not).	Explanation present but lacks detail.	Explanation vague or incorrect.	No explanation.

---

4. Report Quality (20 points)

CRITERION	EXCELLENT (20–18)	GOOD (17–15)	SATISFACTORY (14–12)	NEEDS IMPROVEMENT (11–0)
Screenshots	Clear screenshots of terminal showing successful runs for Exercises 1 and 2 (with at least 4 processes).	Screenshots present but not clearly labelled or missing one run.	Screenshots missing or unclear.	No screenshots.
Code inclusion	Source code for Exercise 2 is included in the report, properly formatted.	Code included but formatting is poor.	Code missing or incomplete.	No code.

CRITERION	EXCELLENT (20–18)	GOOD (17–15)	SATISFACTORY (14–12)	NEEDS IMPROVEMENT (11–0)
Answers to questions	All questions (from discussion in lab) answered correctly and thoroughly.	Most questions answered; some lacking depth.	Answers superficial or incomplete.	No answers.
Reflection	Insightful reflection (10–15 sentences) on challenges and learning from point-to-point communication vs. collectives.	Reflection present but shorter or less insightful.	Reflection minimal or off-topic.	No reflection.

---

5. Self-Assessment Questions (10 points)

(If included in the lab, otherwise points redistributed.)

- Each of the 5 self-assessment questions is worth 2 points, graded for correctness and completeness.

Final Grade Calculation

- Sum points from all categories (max 100 + 10 bonus).
- Bonus points can raise the total above 100, but final grade will be capped at 100.

Note: Submit your report as a **single PDF** with all required content. Ensure your code is properly formatted (use a monospaced font or code blocks). Late submissions will incur a penalty as per course policy.